



OOP Foundations & Keywords

Brief Overview

This note covers **object-oriented programming** and was created from the [OOP 1 | Introduction & Concepts - Classes, Objects, Constructors, Keywords](#) YouTube video. It introduces classes and objects, explains constructors and the `this` keyword, discusses memory allocation (heap vs. stack) and touches on garbage collection and finalization.

Key Points

- Core OOP ideas: classes, objects, encapsulation, inheritance, polymorphism.
- How to instantiate objects with `new` and work with reference variables.
- Role of constructors, overloads, and the `this` keyword in initializing state.
- Basics of Java memory model, including heap/stack, `final`, and garbage collection.



OOP Syllabus Overview

 Topic	 Brief Description
Packages	Grouping related classes.
Static / Non-static	Class-level vs. instance-level members.
Singleton Class	Ensures a single instance.
Properties	Attributes of a class (e.g., roll number, name).
Polymorphism	Same interface, multiple implementations.
Overloading / Overriding	Same method name with different signatures vs. redefining a superclass method.
Constructor & Special Methods	Object initialization and lifecycle hooks (<code>this</code> , <code>final</code> , <code>finalize</code>).
Inheritance Types	Single, multiple (via interfaces), hierarchical, etc.
Encapsulation	Hiding internal state behind methods.
Abstraction	Exposing only essential features.
Access Control	<code>public</code> , <code>protected</code> , <code>private</code> , <code>package-private</code> .
Object Class & Cloning	Root of all classes; copying objects.
Interfaces & Abstract Classes	Contracts vs. partial implementations.
Generics	Type-safe collections and methods.
Exception Handling	<code>try</code> , <code>catch</code> , <code>finally</code> , custom exceptions.
Collection Framework	Lists, sets, maps, queues.
Lambda Expressions	Functional style for interfaces.
Enums	Fixed set of constants.
File Handling	Reading/writing files.
Fast I/O	Buffered streams for performance.

Classes and Objects

Class – A *named group of properties (fields) and functions (methods)* that defines a new data type.

Object – An *instance of a class*; the concrete entity that occupies memory.

- By convention, class names start with a capital letter (e.g., Student).
- **Real-world analogies**
 - **Car** → Car class defines properties like engine, price, seats. Specific models (BMW, Audi, Ferrari) are *objects* with different values.
 - **Human** → Human class defines properties such as arms, legs, eyes. Each person is an *object* of that class.

Instance Variables & the Dot Operator

Instance Variable – A variable declared **inside a class** (but **outside** methods/constructors) that belongs to each object of that class.

- Access syntax: `objectReference.instanceVariable`
 - Example: `student1.rollNumber` returns the roll number of `student1`.
- The **dot (.)** acts as a *separator* linking a reference variable with one of its instance variables or methods.

Creating Objects

new Operator – Dynamically allocates memory **at run time**, creates an object on the **heap**, and returns a reference to it.

```
Student student1 = new Student(); // allocate a Student object, store its reference in student1
```

- **Reference Variable** (e.g., `student1`) lives in **stack memory**; it holds the address of the actual object on the **heap**.
- Before `new` is used, a reference variable holds the default value **null** (no object assigned). Attempting to use it results in a compile-time error: *“variable might not have been initialized.”*

Object Properties

Property	Meaning
State	Current values of the object's instance variables (e.g., <code>rollNumber = 14</code>).
Identity	The unique location of the object in memory; two references can point to the same object.
Behavior	The actions the object can perform, defined by its methods (e.g., <code>greet()</code> for a Human).

- **Identity Example:** Two reference variables pointing to the same Student object will see changes made through either reference.

Packages (Brief)

- Packages organize classes into namespaces, preventing naming conflicts and providing access control.
- Syntax (mentioned later): `package com.example.myapp;`

Heap vs. Stack Memory

Heap – Region of memory where **objects** are allocated at **runtime**.

Stack – Region of memory that stores **reference variables** and primitive values created inside methods.

- Reference variables (e.g., `student1`) live on the **stack** and hold the **address** of an object residing in the **heap**.
- The actual memory address is **not accessible** in Java; this protects against pointer arithmetic errors.

Location	Stores	Lifetime
Stack	Primitive values, reference variables	Created when a method is entered, discarded when it exits
Heap	Objects (<code>new Student()</code>)	Exists until no reference points to it (garbage-collected)

Dynamic Memory Allocation & the new Operator

Dynamic memory allocation – Allocation of memory **during program execution** (runtime) rather than at compile time.

- Code on the **left side** of `=` (declaration) is processed **at compile time**.
- The expression on the **right side** (`new Student()`) is executed **at runtime**, causing the JVM to allocate space on the **heap** and return a reference.

```
Student student1 = new Student(); // heap allocation, stack reference
```

- The `new` keyword **creates** an object and **initializes** its memory with default values (zero, null, 0.0, etc.).

Reference Variables and Default Values

Reference variable – A variable that stores a **reference** (address) to an object in the heap.

- Before `new` is invoked, a reference variable holds null.
- After allocation, the object's fields contain **default values** until explicitly set:

Data type	Default value
<code>int</code> / <code>byte</code> / <code>short</code> / <code>long</code>	0
<code>float</code> / <code>double</code>	0.0
<code>boolean</code>	false
<code>char</code>	'\u0000' (null character)
Object types (<code>String</code> , custom classes)	null

```
System.out.println(Kunal.rollNumber); // prints 0
System.out.println(Kunal.name);       // prints null
System.out.println(Kunal.marks);      // prints 0.0
```

The this Keyword

this – A reference to the **current object** whose members (fields, methods) are being accessed.

- Used inside constructors and instance methods to disambiguate between **parameters** and **instance fields**.
- Equivalent to Python's self; automatically replaced with the name of the reference variable (Kunal, Rahul, ...) at runtime.

```
public Student(int rollNumber, String name, float marks) {
    this.rollNumber = rollNumber; // instance field ← parameter
    this.name       = name;
    this.marks      = marks;
}
```

Constructors

Constructor – A special **method-like** block that runs **once** when an object is created, initializing its state.

- **Default (no-arg) constructor**: Provided implicitly if no constructor is declared; assigns default values.
- **Parameterized constructor**: Defined by the programmer to require initial values.

Key points

- No return type (not even void).
- Name **exactly matches** the class name.
- Can be overloaded (multiple constructors with different parameter lists).

```
public class Student {
    int rollNumber;
    String name;
    float marks;

    // Default constructor (implicit if omitted)
    public Student() { }

    // Parameterized constructor
    public Student(int rollNumber, String name, float marks) {
        this.rollNumber = rollNumber;
        this.name       = name;
        this.marks      = marks;
    }
}
```

Example: Student Class with Methods

```
public class Student {
    int rollNumber;
    String name;
    float marks;

    // Parameterized constructor
    public Student(int rollNumber, String name, float marks) {
        this.rollNumber = rollNumber;
        this.name       = name;
        this.marks       = marks;
    }

    // Instance method: greeting
    public void greeting() {
        System.out.println("Hello, my name is " + this.name);
    }

    // Instance method: changeName
    public void changeName(String newName) {
        this.name = newName;
    }
}
```

Usage

```
Student Kunal = new Student(13, "Kunal Kushwaha", 88.5f);
Kunal.greeting();           // → Hello, my name is Kunal Kushwaha

Kunal.changeName("Shoe Lover");
Kunal.greeting();           // → Hello, my name is Shoe Lover
```

- The constructor sets all fields at **object creation**, eliminating repetitive assignments (Kunal.rollNumber = 13; etc.).
- Inside greeting() and changeName(), this.name refers to the **specific object's** name field, ensuring each instance behaves independently.

Using the **this** Keyword

Definition – this is a reference to the **current object** whose members are being accessed.

- Disambiguates between **instance variables** and **parameters** that share the same name.
- Example when parameter names match fields:

```
public Student(int rollNumber, String name, float marks) {
    this.rollNumber = rollNumber;
    this.name       = name;
    this.marks       = marks;
}
```

- If the parameter name differs, this is optional but often kept for readability:

```
public Student(int rno, String n, float m) {
    rollNumber = rno; // same effect as this.rollNumber = rno;
    name       = n;
    marks      = m;
}
```

- Inside a method, this behaves the same way, e.g., this.greeting() refers to the method of the current object.

Constructor Overloading & this() Calls

Definition – Constructor overloading provides multiple ways to instantiate an object by defining several constructors with different parameter lists.

- **No-arg constructor** (default) can be explicitly defined:

```
public Student() { }
```

- **Parameterized constructors** require arguments:

```
public Student(int rollNumber, String name, float marks) {
    this.rollNumber = rollNumber;
    this.name       = name;
    this.marks      = marks;
}
```

- One constructor may delegate to another using this(...):

```
public Student() {
    this(0, "Default", 0.0f); // calls the three-argument constructor
}
```

- If a matching constructor does **not** exist, the compiler reports an error (e.g., trying to instantiate with two arguments when only one- and three-argument versions are defined).

Wrapper Classes for Primitive Types

Definition – Wrapper classes encapsulate primitive values inside objects, giving access to methods and enabling use in collections.

Primitive	Wrapper Class
int	Integer
float	Float
double	Double
char	Character

boolean	Boolean
byte	Byte
short	Short
long	Long

```
int a = 10;           // primitive, stored on the stack
Integer num = new Integer(45); // object, stored on the heap
```

- Wrapper objects provide methods such as `intValue()`, `compareTo()`, `toString()`, etc.
- Autoboxing allows implicit conversion: `Integer n = 5;` is compiled to `new Integer(5)`.

Reference Assignment – Shared Objects

Definition – Assigning one reference variable to another copies the **reference**, not the object itself.

```
Student s1 = new Student(13, "Kunal", 85.4f);
Student s2 = s1;           // s2 points to the same object as s1
s1.name = "Changed";
System.out.println(s2.name); // prints "Changed"
```

- No additional memory is allocated; both variables refer to the same heap object.
- Mutating the object through any reference is reflected when accessed through other references.

The **final** Keyword

Definition – `final` makes a variable, method, or class immutable in the sense that it cannot be reassigned or overridden.

Final Variables (Constants)

- Must be **initialized** at the point of declaration or in a constructor.
- Convention: name in **UPPER_SNAKE_CASE**.

```
final int BONUS_PERCENT = 2; // cannot be changed later
```

Final Fields in Classes

- For **primitive** fields, the value cannot change after initialization.
- For **reference** fields, the reference cannot be changed, but the object's internal state can still be modified.

```
class Example {
    final Student fixedStudent = new Student(1, "Alice", 90.0f);
    // fixedStudent = new Student(...); // compilation error
    void updateName(String newName) {
        fixedStudent.name = newName; // allowed: modifies object's state
    }
}
```

```
}  
}
```

Final Classes

- A class declared final cannot be subclassed.
- Example: public final class Integer { ... } (the wrapper classes are final).

Pass-by-Value vs. Pass-by-Reference

Definition – Java always uses **pass-by-value**; for primitives the value is the actual data, for objects the value is the **reference**.

- **Primitive example** – swapping inside a method does not affect the original variables:

```
void swap(int a, int b) { int temp = a; a = b; b = temp; }  
// calling swap(x, y) leaves x and y unchanged
```

- **Object example** – the reference is passed by value; both the caller and callee refer to the same object, but reassigning the parameter does not affect the caller's reference.

```
void reassign(Student s) { s = new Student(); } // caller's reference unchanged
```

- Wrapper objects (Integer, Float, etc.) are immutable; even though they are objects, their state cannot be changed, so swapping their references behaves like primitive swapping.

Summary of Key Interactions

Concept	How it relates to earlier material
this	Connects constructor parameters to the object's fields discussed in Instance Variables .
Constructor overloading	Extends the Constructor topic by allowing multiple initialization signatures.
Wrapper classes	Provides an object view of primitive data types , bridging the gap between stack-only primitives and heap-based objects.
Reference assignment	Reinforces the Heap vs. Stack relationship; multiple references can point to a single heap object.
final	Adds immutability rules to Access Control and Object State concepts.
Pass-by-value	Clarifies why method arguments behave the way they do, complementing earlier notes on Method Calls .

Garbage Collection Overview

Garbage Collection – An automatic process in Java that reclaims memory occupied by objects that are no longer reachable from any live reference variable.

- Operates on the **heap** where objects reside.
- Runs **implicitly**; developers cannot invoke it directly to free a specific object.
- When an object becomes unreachable (e.g., a reference variable is reassigned), the collector may later reclaim its memory.

Finalization & the finalize Method

Finalization – A mechanism that lets a class specify actions to be performed just before the garbage collector actually removes the object.

- Implemented by overriding the **finalize()** method inherited from `java.lang.Object`.
- Historically used for resource cleanup (e.g., closing a file stream).
- **Deprecated** in recent Java versions; newer APIs (e.g., try-with-resources) are preferred.

How finalize Works

- The JVM identifies an unreachable object.
- Before reclaiming its memory, it places the object in a **finalization queue**.
- The `finalize()` method of that object is invoked **once** by a dedicated finalizer thread.
- After `finalize()` completes, the object becomes eligible for ordinary garbage collection.

Example finalize Implementation

```
public class A {
    @Override
    protected void finalize() throws Throwable {
        System.out.println("Object of class A is being destroyed");
        super.finalize(); // optional: invoke Object's finalize
    }
}
```

- `@Override` signals that the method replaces `Object.finalize()`.
- `throws Throwable` is required by the method signature (covered later).

Limitations & Best Practices

- **No guarantee of timing** – `finalize()` may run long after the object becomes unreachable, or not at all before the JVM exits.
- **Performance cost** – Objects with finalizers are processed slower because they survive at least one extra GC cycle.
- **Unreliable for critical resources** – Use explicit cleanup (`close()`, try-with-resources) instead of relying on finalization.
- **Cannot be called manually** – Attempting `obj.finalize()` throws an exception; the method is intended solely for the GC.

Triggering Garbage Collection (Demonstration)

- A loop creates many instances of A using a single reference variable:

```
for (int i = 0; i < 10000; i++) {  
    obj = new A();    // each iteration overwrites the previous reference  
}
```

- Each iteration discards the previous object, making it unreachable.
- When memory pressure rises, the garbage collector runs and invokes `finalize()` for the reclaimed objects, printing **“Object of class A is being destroyed.”**

Observations from the Demo

- With a small number of objects, the GC may not run, so no output appears.
- Increasing the allocation count forces the GC to act, and the finalizer messages become visible.
- The same reference variable (`obj`) can point to many different objects over time; only the most recent object remains reachable.

Why Manual Destruction Is Not Allowed

Manual object destruction – The act of explicitly freeing an object's memory by the programmer.

In Java, this is prohibited; only the garbage collector decides when an object's memory is reclaimed.

- Prevents **pointer arithmetic** errors and **dangling references**.
- Guarantees **type safety** and **memory safety** across the JVM.

Interview-Relevant Points

Topic	Typical Question	Key Takeaway
Garbage Collection	<i>How does Java know when to free an object?</i>	Objects become unreachable; GC reclaims them automatically.
<code>finalize</code> method	<i>What is the purpose of <code>finalize()</code> and why is it deprecated?</i>	Allows one-time cleanup before GC; deprecated due to unpredictability and performance issues.
Resource Management	<i>What should you use instead of <code>finalize()</code> for closing files?</i>	<code>try-with-resources</code> or explicit <code>close()</code> calls.
Finalizer Thread	<i>Can you force the finalizer to run?</i>	No; you can suggest GC (<code>System.gc()</code>), but finalizer execution remains nondeterministic.